Tracing model-generated DNA with position-independent watermarking

Kimon Antonios Provatas[1,*] Christos Galanopoulos[1,*]
Ilias Georgakopoulos-Soares[1,†]

[1]The University of Texas at Austin, Austin, Texas, USA
[*]These authors contributed equally. [†]Correspondence: ilias@austin.utexas.edu

Abstract. Genomic language models can write synthetic DNA that carries no record of its origin. A generation-time watermark could provide a provenance signal, if a verifier can detect it using only the DNA sequence, a key, and published detector settings, without knowing where the generated region starts, which strand it lies on, or how it was divided into six-base tokens. We embed the SynthID tournament watermark in two genomic language models, Carbon and GENERator-v2. The verifier searches both strands, every start position, and four window lengths, and corrects its decision for the whole search. In a detection cohort of 1,544 held-out prompts, it found all 3,088 marked sequences per model, unedited and after one substituted, inserted, or deleted base. For ordinary sequences, the one-sided 95% upper confidence bound on the false-positive rate was at most 0.850%; one bound for the wrong-key control reached 1.015% (Carbon, after a deletion). In a development cohort, neither model showed a detectable change in likelihood or in predefined sequence measures. Under random edits placed without reference to the detector, detection stayed complete or nearly complete up to a 2% per-base edit rate. Every test sequence had the same layout, a 384-base prompt followed by a 3,072-base generated region, so other region lengths and placements remain untested. These results show statistical detection, not biological function, secret-key security, or robustness to an editor who sees the detector.

1 Introduction

Genomic language models now generate coding sequences, regulatory elements, genome-scale constructs, and functionally constrained DNA designs [31, 38, 23]. A DNA sequence does not record whether it came from an organism, a design process, or a generative model, and that provenance is lost once the sequence leaves the system that produced it. One biosecurity proposal for designed proteins is to store synthetic gene sequences and synthesis data in repositories that are queried only in emergencies [3]. A provenance signal carried by the sequence itself would complement such records, because it cannot be separated from the sequence.

Watermarking embeds a statistical signal during generation, and a verifier holding the key can later test for it without the model, the prompt, or the generation logs [24]. In DNA, the verifier must first find where to look. Many genomic models write DNA as non-overlapping six-base tokens, but the emitted sequence does not show where one token ends and the next begins. We call the offset of that division the token phase. There are six phases, and the term is unrelated to alignment against a reference genome. Reverse complementation changes the tokens and their order, so both strands must be considered. When the generated region sits inside a longer sequence, its start and end are also unknown. The verifier has to search all of these possibilities and still control the probability of a false positive for the sequence as a whole.

We integrate the SynthID tournament watermark [9] into two genomic language models, Carbon-500M [4, 21] and GENERator-v2 1.2B [27, 12]. Both write six-base tokens and can sample directly from the same vocabulary. They are not independent, because the Carbon training corpus is built mainly on GENERator-v2's pre-training data [4]. SynthID preserves the model's output distribution in expectation over fresh keyed functions, and at a fixed key and context it samples from a deliberately reweighted distribution. We ask whether the sampler follows that distribution, whether watermarking changes likelihood or simple sequence statistics, and whether a verifier can find the mark without knowing the start, strand, or token phase.

The study uses two cohorts of prompts with separate roles. A development cohort of 256 prompts serves the sampler check, the quality comparison, and the choice of how to score a window. A detection cohort of 1,608 prompts, of which 1,544 are held out, serves detection, the false-positive rate, and the edit experiments.

In both models, the verifier found every marked sequence in the detection cohort, unedited and after one edited base. For ordinary sequences, every one-sided 95% upper confidence bound on the false-positive rate lay below the 1% target. Neither model showed a detectable change in the quality measures. Random edits placed without reference to the detector left detection complete or nearly complete up to a 2% per-base edit rate. After an insertion or deletion, the windows the verifier selected are consistent with regions where the token phase is preserved or recovered.

2 Background

2.1 Watermarking designed DNA

DNA watermarks predate generative models. The first synthetic bacterial genome carried watermark sequences

[13], and a short signature was written into the genome of an engineered *Bacillus subtilis* strain [2]. DNA-Crypt hid messages in synonymous codons [18], and messages placed in an open reading frame without changing the encoded protein persisted during viral replication [28]. These designs need a location for the payload and attention to the biological effect of each change. Generation-time watermarking addresses a complementary setting by embedding the signal through the model's sampling distribution during generation, so it needs no payload region.

2.2 Watermarking language-model output

Language-model watermarks change how tokens are sampled. Some bias a pseudorandom subset of tokens and detect its excess frequency [24]. Others leave the output distribution unchanged in expectation over fresh keyed randomness [7, 26, 1, 20, 39]. SynthID in its non-distortionary configuration belongs to the second group. We apply it to DNA and propose no new construction. Robustness depends on what the attacker can do, and prior work describes evasion, key-recovery, and forgery attacks by stronger and adaptive attackers [40, 11, 36, 22, 35]. This study tests none of them; its edits are random and never consult the detector.

Text watermarks built to survive editing treat synchronization as a local problem. Kuditipudi et al. align a candidate text to a key sequence under an edit distance [26], and Kirchenbauer et al. search all contiguous spans and calibrate the false-positive rate for that search [25]. Both rely on an edit disturbing tokenization only near the edit. Golowich and Moitra tolerate a constant fraction of adversarial edits, but with a new construction built on pseudorandom codes [14]. A fixed vocabulary partition keeps the key synchronized under a positional shift, which roughly doubles the provable edit tolerance relative to the soft watermark keyed on the previous token without making the mark immune to edits [42]. Watermarks on sentence embeddings avoid tokenization but need an encoder and a natural segmentation into units [19], which a DNA verifier lacks.

Six-base tokens remove the locality these methods rely on. One inserted or deleted base shifts the reading frame, so every later token changes identity, and so does the keyed context that sets each mark bit. Alignment cannot repair this, because no token sequence survives to be aligned, and a fixed partition does not help, because the changed tokens carry independent mark bits. What survives is a contiguous region read in one of six phases. Verification therefore becomes a search over strands, phases, start positions, and window lengths, and the correction for that search is part of the method.

Four properties separate these constructions here: whether verification needs the generating model; whether it needs an alignment, a known boundary, or a known strand; which edits it claims to tolerate; and whether sampling is distorted. Biased-logit schemes distort sampling and are usually verified with known boundaries [24]. Distribution-preserving schemes realign to a key sequence [26] or certify a radius of token changes [39]. Watermarks for biological sequence assume a coding region [41] or mark the protein, not its DNA [5]. The construction evaluated here needs no model, alignment, known boundary, or coding region, and it preserves the sampling distribution in expectation.

2.3 SynthID

We use the binary tournament construction of SynthID-Text [9]. At each generation step, a secret key and the most recent tokens assign every candidate token a binary mark bit in each tournament layer, and each layer favors candidates whose mark bit is 1. Averaged over fresh keyed functions, the tournament leaves the next-token distribution unchanged. At a fixed key and context, it samples from a reweighted distribution. The detector rebuilds the mark bits from the output and tests for an excess of 1s. Because the bits depend on local context, not on absolute position, they can be rebuilt from any retained stretch of sequence, and the detector needs only the output, the key, and published detector settings (Section 3.3). Keying on recent tokens is common to autoregressive text watermarks [24]. We chose SynthID because it combines local keying with tournament sampling and has been evaluated in a large-scale text deployment.

2.4 Genomic tokenization and token phase

Several genomic models, including both studied here, divide DNA into non-overlapping six-base tokens, which shortens the token sequence and lets a model cover more bases in a fixed context [8, 38, 4, 27]. Models with one token per base have no phase problem, because an insertion or deletion moves tokens without changing them [31, 32]. Caduceus uses one token per base because small input changes can otherwise change the tokenization drastically [37]. GENERator randomizes the tokenization offset of each training sample [38], and GENERator-v2 cycles through all six offsets [27]. None of these remedies is available to a verifier that receives only DNA, which must recover the phase from the sequence itself.

3 Methods

3.1 Models and sampling

Generation used Carbon-500M [4, 21] and GENERator-v2-eukaryote-1.2b-base [27, 12], two decoder-only models. At every step, we restricted the logits to the 4,096 canonical DNA tokens and normalized once. For GENERator, this is its direct token distribution, not the per-base sampling that its released code applies by default. Temperature was 1.0, with no top-k or top-p truncation. The resulting distribution p is shared by both samplers: the ordinary sampler draws from p without a key, and the marked sampler applies the tournament below. We report each model separately.

3.2 Watermark embedding

At each step, the key, the preceding four tokens, and a candidate token determine the candidate's mark bit g, computed with HMAC-SHA-256 over the key, a fixed domain label, the context, and the candidate. The tournament formula, the number of layers, the context width, and the repeated-context rule follow the public reference implementation of SynthID-Text [15]. One tournament layer reweights the distribution as

$$p'(x) \;=\; p(x)\,\Big(1 + g(x) - \textstyle\sum_y p(y)\,g(y)\Big), \qquad (1)$$

so candidates with a mark bit of 1 gain probability by an amount set by the total probability of such candidates. The sampler applies $m = 30$ layers, each with its own mark bits, and samples from the resulting distribution p_k (Figure 1a).

The first four generated tokens are sampled ordinarily, so every later context can be rebuilt from the continuation alone. A four-token context that has already occurred is also sampled ordinarily and excluded from the detector score, as in the original construction [9]; the sampler keeps a history of 1,024 contexts to recognize repeats. Averaged over fresh keyed functions, Equation 1 leaves the next-token distribution unchanged. At a fixed key and context, the sampler follows the reweighted distribution p_k, so the sampler checks test against p_k.

3.3 Position-independent detection

The verifier receives a DNA sequence, the key, and the published detector settings: the generation domain label, the four window lengths, the 30 tournament layers and four-token context used to rebuild the mark bits, the 1,024-context repetition history, and the significance level $\alpha = 0.01$. In deployment, only the key would be secret. It does not receive the prompt, the model or its probabilities, the seed, the strand, the token phase, or the generation boundary (Figure 1b).

The verifier treats every candidate generated region as a window: a stretch of the sequence on one strand, starting at a given base, with one of four lengths. It scores windows at every starting base of the sequence and of its reverse complement, which covers every start position and all six token phases. Inside a window, the DNA is divided into tokens from the window's first base, and the first four tokens supply context. Every later token whose context is not a repeat is a scored token. The verifier rebuilds the mark bits of the scored tokens and counts the ones, S, among their n mark bits.

If the sequence carries no mark for this key, the mark bits behave as independent fair coin flips, and the window P-value is the binomial tail

$$P_{\text{win}} \;=\; \Pr\big[\text{Binomial}(n, \tfrac{1}{2}) \geq S\big]. \qquad (2)$$

For a sequence of N bases, the verifier scores

$$M \;=\; 2\sum_{L}(N - L + 1), \qquad L \in \{384,\ 768,\ 1{,}536,\ 3{,}072\}, \qquad (3)$$

windows, which is 16,136 for a 3,456-base sequence, and makes one decision for the sequence:

$$\begin{aligned} P_{\text{seq}} \;&=\; \min\big(1,\ M \cdot \min_{\text{windows}} P_{\text{win}}\big), \\ \text{mark found} \;&\iff\; P_{\text{seq}} \leq \alpha. \end{aligned} \qquad (4)$$

The Bonferroni factor M covers every window searched, including overlapping ones, and the level α was set in advance. We call $-\log_{10} P_{\text{win}}$ the strength of a window. A sequence is positive exactly when some window reaches strength $\log_{10}(M/\alpha) = 6.21$. The search depends only on the sequence, so it is the same for both models, and M is recomputed when an edit changes the sequence length. Supplementary Section S4 gives the cost of the search.

Bonferroni correction needs no independence between tests, but it is conservative here, because windows of the same length whose starts differ by a multiple of six bases share a token phase and nearly all their tokens (Supplementary Section S3).

A window need not be intact to be detected. A token that has lost its phase or its context contributes unbiased bits and only dilutes the count, so a window is positive once it holds enough tokens that kept both. In the shortest window, about 56% of the mark bits must be 1 (Supplementary Section S2).

3.4 Prompts and cohorts

Prompts are 384-base forward-strand windows from 16 accession-versioned RefSeq chromosome records of eight model organisms. Windows were chosen by hashing a fixed label, the accession, and a segment index. Human, patient, controlled-access, organelle, plasmid, and ambiguous-base spans were excluded. Each cohort was fixed before any model generated from it, and no prompt was added, moved, or removed afterward.

The development cohort contains 256 prompts. Before any detection result was examined, a hash rule reserved 64 of them for choosing how to summarize the mark bits in a window (Supplementary Section S1) and held out the other 192. The sampler check and the sequence measures use this cohort only.

The detection cohort contains 1,608 prompts, 134 non-overlapping windows from each of 12 of the 16 records, covering six of the eight organisms (not mouse or budding yeast), drawn with a separate selection label. The same hash rule set aside 64 prompts, which were not used, and held out 1,544. A separate generation domain label gave both models new keyed functions, and all other settings matched. The same authors generated and scored this cohort with the same code, in a separate computing environment.

Every prompt produced two marked and two ordinary continuations of 512 tokens (3,072 bases) per model. The two draws used different seeds, and the two marked draws used different keys, both published for reproducibility. Prepending the prompt to a held-out continuation gives the 3,456-base test sequence that the verifier receives, either a marked sequence or an ordinary sequence; its generation boundary is withheld. Every test sequence therefore has the same layout: the generated region is its last

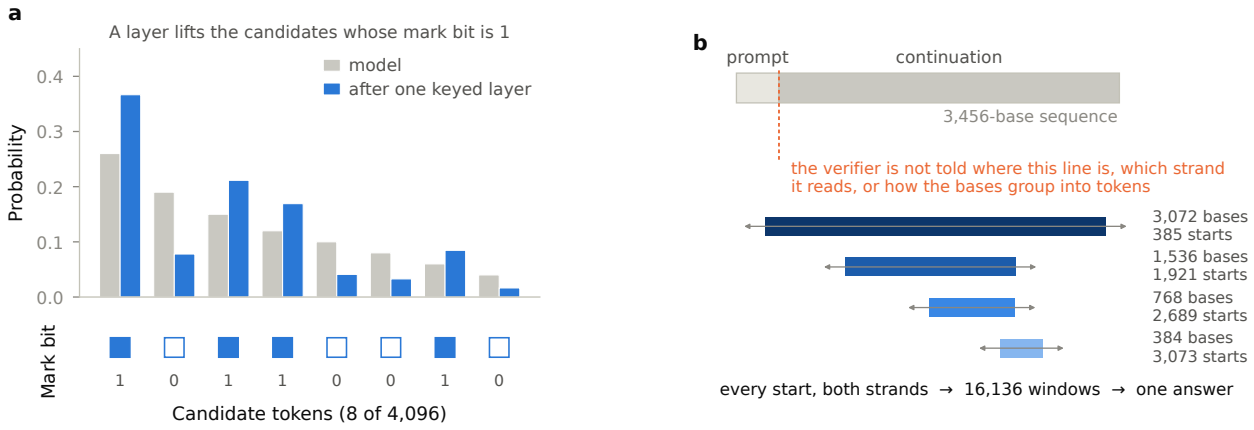


Figure 1: SynthID embedding and position-independent detection. **a**, One keyed layer, drawn for eight of the 4,096 candidate tokens with an illustrative distribution. Candidates whose mark bit is 1 gain probability, and the others lose it. The sampler applies 30 such layers. Panel **a** illustrates Equation 1 and is not a measurement. **b**, Detector inputs and search. The verifier receives the sequence, the key, and the published detector settings, without the generation boundary, the strand, or the token phase. It scores windows of four lengths at every starting base on both strands, 16,136 windows for a 3,456-base sequence, and reports one sequence-level result corrected for the full search.

3,072 bases, lies on the forward strand, and starts a multiple of six bases from the first base (Section 6). Scoring each marked sequence with the other marked draw's key gives the wrong-key control, which tests whether a positive depends on the correct key. The three rates are reported separately, because a pooled accuracy would depend on how common marked sequences are in an application.

3.5 Sampler validation

The correctness of the marked sampler rests on implementation tests in the project's test suite. The tournament update reproduces the public SynthID-Text reference implementation [15] on peaked, flat, random, and degenerate distributions. The vectorized computation of p_k used during generation agrees with an independent unvectorized implementation to floating-point precision. Computing p_k once and applying its inverse-CDF sampling step reproduces the generation sampler's draws exactly.

A second check uses realistic model states. For each model and development prompt, we fixed the state at token index 64 of the first ordinary continuation and drew 5,000 samples from each of p and p_k. Each set of counts was compared with its source distribution by the likelihood-ratio (G) statistic. An expected count of about 1.2 per category is too low for the asymptotic χ^2 reference, so G was referred to a Monte Carlo null of 999 samples from the same distribution, which is exact by construction. Because the draws come from the computed distributions, this check confirms the sampling step and the test, not the computation of p_k. The 256 tests per sampler form one family under Benjamini–Hochberg correction at 0.05. Bonferroni correction cannot reject here, because 999 replicates allow no P-value below 10^{-3}, above its level of 1.95×10^{-4}. As a negative control, at eight further states per model, draws from p were tested against p_k.

3.6 Sequence measures

The model score is the teacher-forced negative log-likelihood per generated token under the same model and token restriction as generation, with the prompt as context only. Each model scores its own outputs, so scores are not compared between models. Thirteen further measures were specified before analysis: GC content; purine content; CpG content; base, dinucleotide, and trinucleotide entropy; distinct 6-mer fraction; longest and mean single-base run; Jensen–Shannon drift from the prompt at $k = 1, 2, 3$; and perplexity. All 14 measures form one family under Benjamini–Hochberg correction.

3.7 Edits and statistics

In the single-edit conditions, each sequence has exactly one substituted, inserted, or deleted base in the continuation, placed by a fixed pseudorandom rule that never consults the verifier. Prompts are the independent unit, and a prompt's two draws are paired repetitions. Uncertainty in the sequence measures used an equal-weight bootstrap over prompts with 20,000 replicates and a paired sign-flip test with 100,000 replicates. A control prompt is positive if either draw is positive; for marked prompts, we also require both draws to be detected.

The detection cohort's analysis plan, specified before generation, reports the exact one-sided 95% upper confidence bound (Clopper–Pearson) on the false-positive rate over the 1,544 held-out prompts. Ordinary sequences, the false positives that matter in use, are the primary control, and the wrong-key control is a secondary check of key specificity. The prompt-level rate is about twice the per-sequence rate that α controls, so comparing it with

the 1% target is conservative. Development-cohort rates carry exact two-sided 95% intervals.

3.8 Edit-rate experiment

To find where random editing defeats the verifier, we rescored the detection cohort's stored sequences after applying $k = \mathrm{round}(3{,}072\,r)$ edits to each continuation. We call r the per-base edit rate and tested seven values from 0.03% to 10% (Figure 5). A substitution changes one base. Each insertion or deletion adds or removes one to five bases, so the edit rate counts edit events, not changed bases. Four series were run: substitutions, insertions only, deletions only, and a mixed series of random insertions and deletions. The single-direction series keep opposite frame shifts from cancelling. Positions and indel lengths came from a public hash stream set by the series, the sequence, and the rate, never by the detector, and no sequence or detector setting changed. Marked sequences were scored with the correct key and ordinary sequences with the corresponding key, 3,088 sequences per model, rate, and series, without wrong-key controls. The analysis plan required the ordinary-sequence rate to stay at or below 1% in every cell, because a rise with edit rate would mean that the edits themselves create apparent signal.

4 Results

4.1 Sampler fidelity

At the tested model states, draws from the fixed-key and model distributions agreed with those distributions, and the test could tell the two distributions apart. Each model had 256 tested states, and chance alone predicts 12.8 of them to reject at $P < 0.05$. The marked sampler produced 13 in Carbon and 5 in GENERator, and the ordinary sampler produced 12 and 14. In GENERator, neither sampler had a rejection after Benjamini–Hochberg correction. For Carbon, the corrected count was not retained with the evidence, and its nominal counts are close to chance. The negative control, draws from the model's distribution tested against the fixed-key distribution, was rejected at all eight control states in each model. Together with the implementation tests (Section 3.5), these results support the correctness of the sampler at the resolution of 5,000 draws per state. They do not prove it correct at every possible state.

4.2 Sequence-quality proxies

Watermarking produced no statistically detectable difference between marked and ordinary continuations in either model (Figure 2). Every interval for the 14 measures contained zero, and no comparison was significant after correction. The smallest adjusted P-value was 0.85 in Carbon and 0.73 in GENERator. The paired difference in model score was +0.00852 nats per token in Carbon and −0.00564 in GENERator. The largest absolute standardized effects were 0.084 standard deviations in Carbon, on the longest single-base run, and 0.111 in GENERator, on the mean single-base run. These results indicate no measurable quality loss under the evaluated proxies.

4.3 Detection and false-positive rate

In the detection cohort, the verifier found every marked sequence in both models, unedited and after each single-base edit (Table 1; Figure 3). That is 3,088 sequences per model and condition, and it places the exact two-sided 95% interval for the prompt-level detection rate at 99.76–100%.

Every one-sided 95% upper confidence bound on the false-positive rate for ordinary sequences lay below the 1% target. The largest was 0.850%, for GENERator after a deletion. The analysis plan had stated before generation that a bound below 1% required at most eight positive prompts.

The wrong-key control, a secondary check of key specificity, crossed 1% in one cell. For Carbon after a deletion, its bound was 1.015%.

The correction was conservative in practice. Per sequence, the corrected decision was positive for 0.105% of ordinary Carbon sequences and 0.186% of ordinary GENERator sequences, pooled across the four conditions, well below α (Supplementary Section S3).

The development cohort gave the same result. All 384 marked sequences per model were found in every condition, and each control produced at most one positive sequence per model and condition (Supplementary Table S1).

Typical detection strength was the same in both models. The median unedited marked sequence reached strength 792.9 in Carbon and 799.7 in GENERator, far above the threshold. The weakest detected sequences differ more, but in the development cohort that difference came from a single GENERator sequence with a highly repetitive continuation (Supplementary Section S6).

4.4 Detection after single-base edits

One inserted or deleted base cost a sequence almost half of its strength, and one substituted base cost little, yet every marked sequence stayed detectable (Figure 4a,c). A substitution lowered the median strength by about 2%, and an insertion or deletion by about 45% (Supplementary Table S2).

An insertion or deletion also changed which window carried the evidence. For unedited and substituted sequences, the full 3,072-base window was strongest in nearly every sequence. After an insertion or deletion, a shorter window was strongest in almost half of the sequences (Figure 4b,d). A substitution changes token content and nearby contexts but keeps the downstream token phase. An insertion or deletion shifts the division of every later token in a window that spans the edit. The shift toward shorter windows is consistent with the search selecting regions whose token phase is preserved or recovered. The development cohort showed the same shift (Supplementary Figure S2).

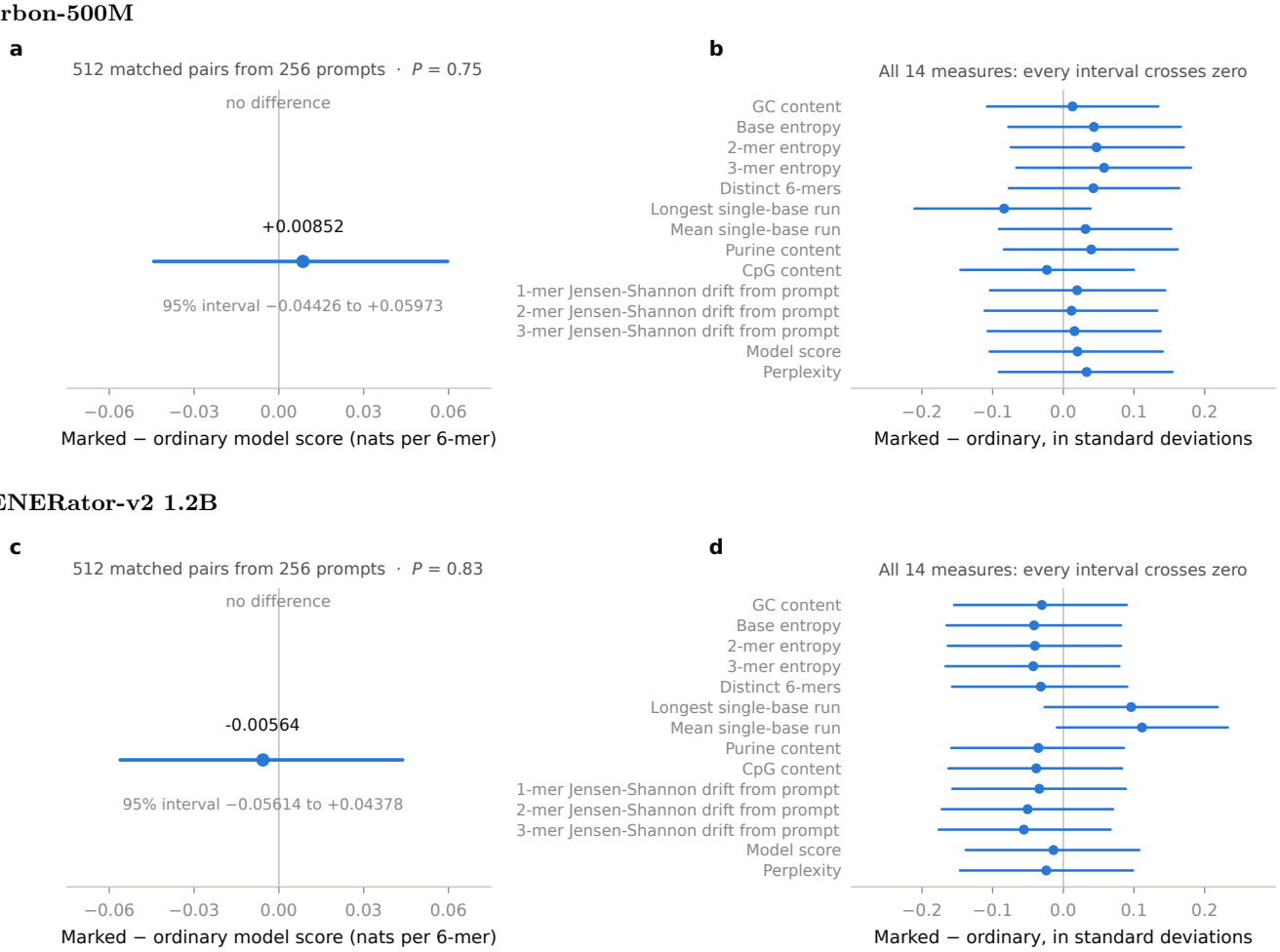


Figure 2: Model likelihood and sequence-quality proxies in the development cohort. a,c, Paired difference in model score between marked and ordinary continuations, over 512 matched pairs from 256 prompts. Positive values indicate lower model likelihood for marked continuations. The horizontal bar is the 95% interval from a bootstrap over prompts: −0.04426 to +0.05973 in Carbon ($P = 0.75$) and −0.05614 to +0.04378 in GENERator ($P = 0.83$). **b,d**, Paired differences for all 14 measures in standard deviations, with intervals in the same units. Every interval includes zero, and none of the 14 comparisons is significant after multiple-testing correction. Panels **a,b** show Carbon and panels **c,d** show GENERator.

Table 1: Positive detections in the detection cohort, for each model across 1,544 held-out prompts with two draws each (3,088 sequences per model and condition). Correct-key entries count marked sequences. Control entries count positive sequences, each from a different prompt; the value in parentheses is the exact one-sided 95% upper confidence bound on the prompt-level rate, in percent. Ordinary sequences were designated in advance as the primary false-positive measure, and the wrong-key control is a secondary check of key specificity. Each edit changes exactly one base.

	Carbon-500M			**GENERator-v2 1.2B**		
Edit applied to the sequence	Correct key	Ordinary	Wrong key	Correct key	Ordinary	Wrong key
None	3,088/3,088	3 (0.501)	5 (0.680)	3,088/3,088	5 (0.680)	2 (0.407)
One substitution	3,088/3,088	3 (0.501)	5 (0.680)	3,088/3,088	5 (0.680)	2 (0.407)
One insertion	3,088/3,088	3 (0.501)	6 (0.766)	3,088/3,088	6 (0.766)	2 (0.407)
One deletion	3,088/3,088	4 (0.592)	9 (1.015)	3,088/3,088	7 (0.850)	3 (0.501)

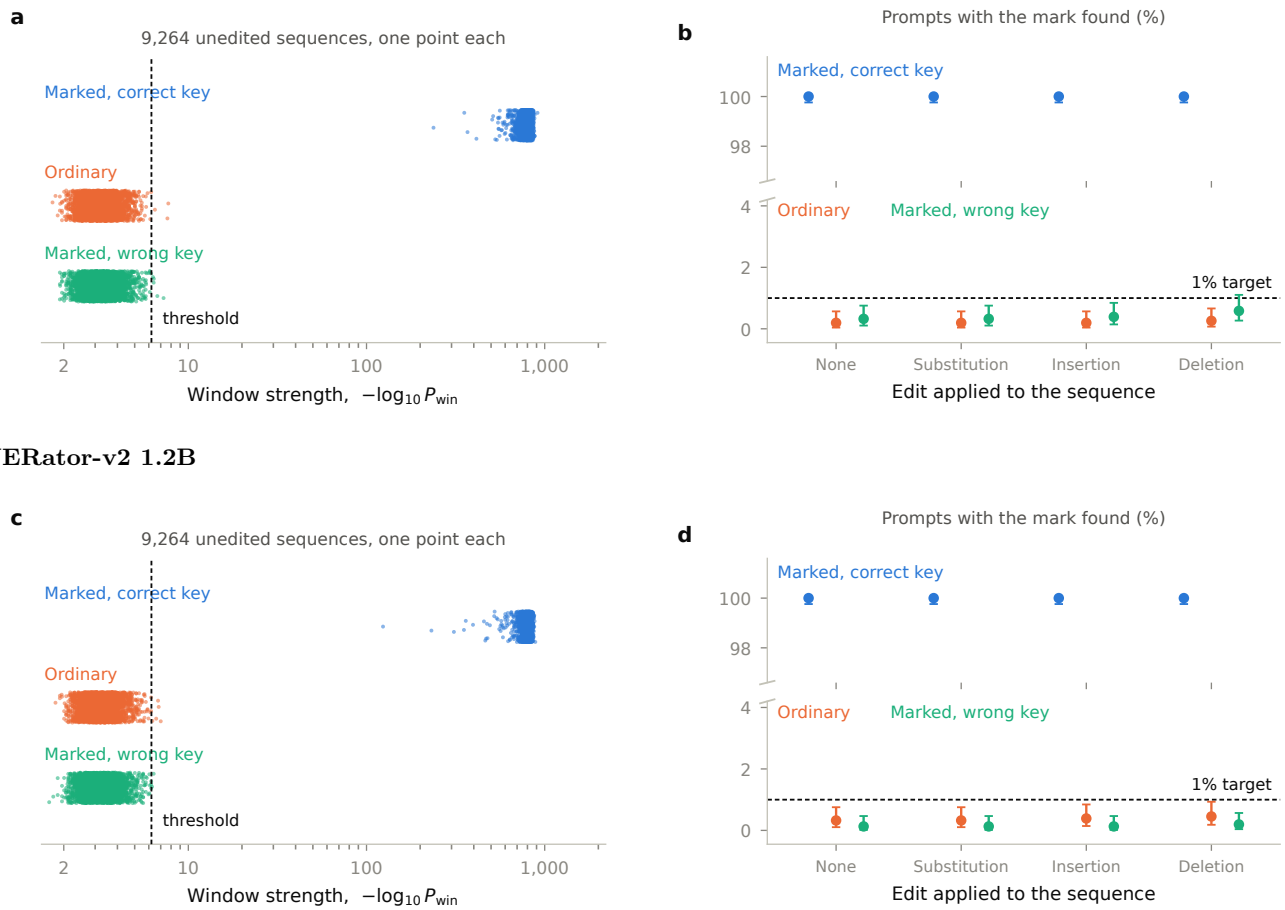


Figure 3: Position-independent detection in the detection cohort. a,c, Strength of the best window in each unedited sequence, one point per sequence, on a log scale: 3,088 marked sequences scored with the correct key, 3,088 ordinary sequences, and 3,088 wrong-key controls per model. The dashed line is the threshold corrected for the search over 16,136 windows. All marked sequences exceed it. **b,d**, Proportion of the 1,544 held-out prompts with positive detections (both draws for marked sequences with the correct key, either draw for controls), with exact two-sided 95% intervals, on a broken axis. All marked prompts were detected in all four edit conditions. Table 1 gives the counts and the one-sided upper bounds used for the false-positive comparison, and Supplementary Figure S1 shows the development cohort. Panels **a,b** show Carbon and panels **c,d** show GENERator.

4.5 Detection as edits accumulate

Detection stayed complete or nearly complete through a 2% per-base edit rate, that is, 61 edits per continuation, in both models (Figure 5). Every marked sequence was found at every rate up to 2% for every edit type, except for one GENERator sequence in the mixed series at 2%.

Insertions and deletions defeated the verifier at lower rates than substitutions. At a 5% rate, substitutions were still detected in every Carbon sequence and all but one GENERator sequence, whereas detection after insertions and deletions fell to 92.9–93.9%. At 10%, substitutions were detected in 19.7% of Carbon sequences and 18.7% of GENERator sequences. The other edit types fell to 4.6–5.9%.

The two models' curves agreed within one percentage point at every rate and edit type, so the single GENERator miss at 2% does not indicate a difference in robustness. The insertion-only and deletion-only series behaved like the mixed series, so cancelling frame shifts do not explain the tolerance.

Ordinary sequences stayed below the 1% target at every rate and in every series. Pooled over all rates and edit types, 0.15% of ordinary Carbon sequences and 0.13% of ordinary GENERator sequences were positive. The largest value in any single cell was 0.29%, for GENERator in the mixed series at 1%.

5 Discussion

A verifier that knows neither the strand, the start of the generated region, nor the token phase can recover a generation-time watermark from DNA, with one decision corrected for the whole search. The verifier is assembled from existing parts: the SynthID detection statistic, a search over contiguous windows of the kind used for text [25], and a Bonferroni correction. What is specific

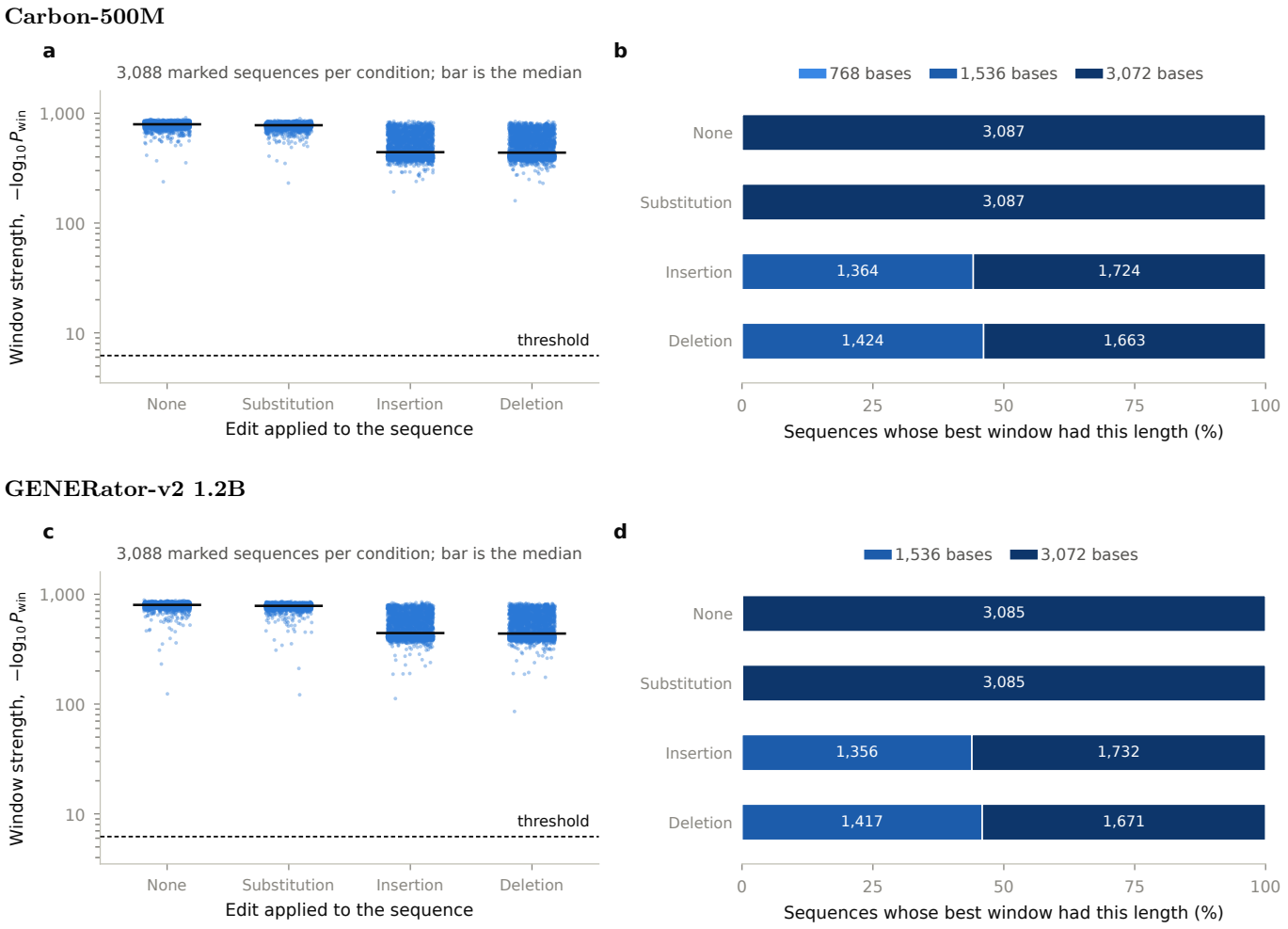


Figure 4: Effects of single-base edits on detection in the detection cohort. a,c, Strength of the best window in each of the 3,088 marked sequences per condition, on a log scale, with the median shown as a bar. The dashed line is the detection threshold. **b,d**, Length of the strongest window; numbers inside the bars are sequence counts out of 3,088. The full 3,072-base window is strongest for nearly all unedited and substituted sequences, whereas shorter windows are selected for almost half of the sequences after an insertion or deletion. Supplementary Figure S2 shows the development cohort. Panels **a,b** show Carbon and panels **c,d** show GENERator.

to DNA is the search space, which adds strands and token phases to start positions and lengths, and the single correction that covers it. This shared watermarking and detection procedure is the methodological contribution.

The verifier tolerates random edits because it does not need an intact window. A simple argument predicts failure much earlier: at a 1% rate, stretches free of insertions and deletions average about 100 bases, shorter than the 384-base minimum window. A window, however, gains excess 1s from every token that keeps its phase and context, and the other tokens only dilute the count. An unedited sequence starts near strength 800 against a threshold of 6.21, so it can lose most of its evidence before the decision changes. The shift toward shorter windows after an insertion or deletion fits the same picture, with the search finding the original phase again downstream of the edit. The strongest window does not identify the exact generation boundaries.

Running the same sampler and verifier in two models shows that the implementation carries across them. The two models agreed in detection, had nearly identical typical strength, and showed no measurable quality loss, although they share much of their training data.

The margin seen here should not be assumed for shorter generated regions or longer sequences. A shorter region supplies fewer mark bits, and a longer sequence adds windows to the correction. A region whose length falls between two window lengths is covered only in part by the shorter window, or shares the longer one with unmarked bases that dilute the count. The margin also varies between sequences. The weakest marked sequences lay well below the median (Supplementary Section S6), and we did not measure how strength depends on the model's per-token entropy. The shortest detectable continuation, and the trade-off between detection power and distortion across tournament depths, remain to be measured in both models.

The search addresses a synchronization problem that coding theory treats directly. Synchronization strings add indexing information to recover from insertions and deletions [16], and pseudorandom codes combine indistinguishability with decoding under specified errors [6].



Figure 5: Detection as random edits accumulate, detection cohort. **a,b**, Percentage of 3,088 marked sequences detected with the correct key in Carbon (**a**) and GENERator (**b**), by per-base edit rate (log scale) and edit type. The rates were 0.03, 0.1, 0.5, 1, 2, 5, and 10%; the top axis gives the corresponding 1, 3, 15, 31, 61, 154, and 307 edits per 3,072-base continuation. Each insertion or deletion adds or removes one to five bases, and in the mixed series each edit is an insertion or a deletion at random. **c**, Percentage of ordinary sequences detected under the corresponding key, pooled over the four edit types (12,352 sequences per point), against the 1% target. Pooled over every rate and edit type, including the unedited sequences, 132 of 89,552 Carbon sequences and 116 of 89,552 GENERator sequences were positive. Vertical bars are exact two-sided 95% intervals and are often smaller than the markers. Edit positions were chosen without reference to the detector.

Davey and MacKay's watermark codes integrate over the accumulated drift with a hidden Markov model, but they presume control of the encoding, which a sampling watermark does not have [10]. Our verifier adds no indexing code to the sequence. It enumerates the offsets, pays for the search through its correction, and carries none of the error-correction guarantees of those codes. Two measurements indicate where a different verifier could gain. The correction is about 19 times more severe than the effective number of tests (Supplementary Section S3). The decision also rests on the single strongest window, and one insertion or deletion removes about 45% of the median strength (Section 4.4). A threshold calibrated to the dependence between windows, or a score that combines windows on both sides of a phase shift, could address these. We evaluated neither. Synonymous-codon watermarks for coding sequences [41] serve a complementary goal, and combining them with generation-time marks would need its own evaluation.

6 Limitations

This evaluation covers two models, one watermark construction, and one verifier, and it gives no general proof of watermark robustness, biological safety, or secret-key security. Both models use six-base tokens; single-base or variable-length tokens would pose different alignment problems. Both cohorts come from the same 16 chromosome records or a subset of them, which limits extrapolation to other genomic sources, and longer sequences would increase the search correction. The models are not independent, because Carbon's training corpus is built mainly on GENERator-v2's pre-training data, and their agreement does not make them equivalent in any respect.

We compared SynthID with its matched ordinary sampler, not with another watermark construction on the same cohort, such as a biased-logit scheme adapted to six-base tokens or another distribution-preserving scheme. That comparison would confound the construction with the verifier that is our contribution, and the property comparison in Section 2 does not substitute for measured performance on a shared cohort. Nor did we compare the full search with narrower detectors at the same false-positive target: a detector told the token phase and the generation boundary, a search over the six phases alone, or a span search adapted from text watermarking [25]. The detection power that each part of the search adds, and whether it repays its share of the correction and of the search cost (Supplementary Section S4), is therefore unmeasured.

All test sequences share one layout. The generated region is always the last 3,072 bases of a 3,456-base sequence, on the forward strand, a multiple of six bases from the first base, and exactly as long as the longest window. We did not place a generated region between two unmarked flanks, crop it, supply the reverse complement, or vary its length, its start, or the length of the surrounding sequence, and no region length between two window lengths was tested. The search covers these cases by construction. Its implementation tests confirm, on a synthetic marked region that is not model output, detection between two unmarked flanks at start offsets that are not multiples of six, and an identical result for the reverse complement. Detection rates for these cases on model output are unmeasured. The insertion and deletion experiments shift the phase of the tokens downstream of each edit, but they do not vary the region's placement or length. How detection depends on the length of the generated region, the length of the flanks, and the region's position therefore remains open, as does the shortest region the verifier can detect.

The sampler and quality results come from one execution per model on the development cohort. The detection cohort was generated and scored in a separate computing environment with new keyed functions, but by the same authors with the same code, so it is not an independent replication. For Carbon, the protocol document retained with the development-cohort detection result does not match the hash recorded in that result, and the original document could not be recovered, so the cause of the mismatch is unknown. The run took its settings from the defaults of the detector code and recorded only a hash of the document. An audit found that eight of the nine settings it governs are recorded in the result's own fields and agree with the reported analysis. The ninth, the 1,024-context repetition history, rests only on the default of the detector code identified by its recorded hash. The mismatch therefore gives no indication that the sequences or the detection statistic differ from what we report, but the execution cannot be checked against the document itself. In the detection cohort, the recorded hash matches the protocol document for both models.

The false-positive evidence is narrow. With 192 prompts, the development cohort could not show a rate below 1% (Supplementary Table S1). In the detection cohort, every bound for ordinary sequences lies below 1%, but the largest, 0.850%, does so narrowly, and the wrong-key bound for Carbon after a deletion exceeds that level at 1.015%. These rates apply to the sequence length and sequence sources tested here; elsewhere, false-positive control rests on the Bonferroni correction alone.

That correction is valid only if each window P-value is valid, which requires mark bits rebuilt without the correct key to behave as independent fair bits. The aligned-detector diagnostic (Supplementary Section S7) checks this for GENERator with the token phase known, not for the full search. We checked the full search only through its realized rates on ordinary sequences and wrong-key controls (Section 4.3; Figure 5c). Those sequences are model output following RefSeq prompts, at essentially one sequence length, scored with the two published keys under one domain label per model and cohort. We did not calibrate the sequence-level P-value across many independent keys or on genomic DNA that no model generated.

Every edit was placed at random, without reference to the detector, and the edit-rate series scored no wrong-key controls. Each insertion or deletion spanned at most five bases at a randomly drawn position. We did not test clustered edits, longer insertions or deletions, recombination of marked and unmarked segments, or cropping, and the results are not broken down by edit position. We did not test an adaptive editor who inspects the detector, the key, or any score and places edits where they do the most damage.

The keys are published for reproducibility, so secrecy, resistance to key recovery, and removal in deployment were not evaluated, and the analytic null assumes independent fair mark bits without the correct key. Attacks reported against text watermarks remain untested here, including key recovery from the detector's P-values [35], paraphrasing and back-translation of SynthID-Text [17], and a layer-inflation attack shown to affect the mean score that our verifier also uses [33].

Model likelihood and sequence statistics are proxies, not measures of expression, replication, function, viability, or safety, and our quality claim covers only the 14 measures. Standardized in-silico suites, such as supervised downstream tasks and zero-shot variant-effect benchmarks [30, 29, 34], score a model's reading of annotated DNA and mostly rely on labels tied to genome coordinates that a new generation does not carry. Evaluating marked outputs against them would need a paired design or an experimental assay, which is the natural next step before any claim about biological consequence.

7 Conclusion

The SynthID tournament watermark can be embedded in Carbon-500M and GENERator-v2 1.2B without a statistically detectable change in the evaluated quality proxies. A verifier holding the key can recover it from DNA by searching strands, start positions, and window lengths under one correction. In the detection cohort, the verifier found every marked sequence in both models, unedited and after one edited base, and the one-sided 95% upper confidence bound on the false-positive rate for ordinary sequences was at most 0.850%. Detection survived random edits up to a 2% per-base rate and failed for most sequences at 10%. Broader conclusions require independent replication, wider sequence coverage, other region lengths and placements, tests against adaptive editing, and biological validation.

AI use statement

Generative AI tools assisted with method implementation, experiment design, statistical derivations, data preparation, literature retrieval, and interpretation of results. They also assisted with drafting and revising the manuscript, articulating the verification setting and claim scope, and preparing the LaTeX submission. Carbon-500M and GENERator-v2 1.2B generated the DNA continuations studied in the experiments, as described in Section 3. The paper contains no formal theorems or proofs. Its statistical development uses standard probability models and a Bonferroni correction, and the authors verified all derivations, numbers, and claims against the recorded measurements. The authors are responsible for the final text, claims, and artifacts.

Ethics statement

The prompt cohorts use public non-human RefSeq records and exclude patient and controlled-access sequences (Section 3.4). The experiments assess statistical detection and sequence proxies. They do not establish biological function, viability, safety, or sequence authenticity. The experimental keys are published for reproducibility and provide no evidence of secrecy or resistance to key

recovery. A detected watermark is evidence under the specified statistical model, not a certificate of biological origin or safe use.

Reproducibility statement

Section 3 specifies the canonical token policy, the tournament update, the detector search and its correction, prompt construction, and the statistical comparisons, and the Supplementary information gives the window-score selection, the search cost, and the computing environment. The figures and tables report both models separately under the same evaluation design. Section 6 records the execution scope, the absence of an independent replication, and the Carbon protocol-document discrepancy together with its audit. The code, protocol documents, and evidence ledger needed to reproduce the analysis are available as described in the Data and code availability statement.

Data and code availability

The verifier implementation, the watermark sampler, the prompt-cohort construction scripts, the analysis and figure scripts, the protocol documents, and the evidence ledger to which every number in this paper resolves are maintained in the project GitHub repository. The repository includes the detection results for both models in the detection cohort and the GENERator results for the development cohort. The Carbon development-cohort result files, generated sequences, and other underlying files not in the repository are available from the authors on request. The sampler's implementation tests, including the comparison with the SynthID-Text reference implementation at the cited commit, are part of the repository's test suite. The two experimental keys are published for reproducibility and are not deployment secrets.

Competing interests

The authors declare no competing financial or non-financial interests.

Acknowledgments

This work has been supported by the National Institute of General Medical Sciences of the National Institutes of Health [R35GM155468 to I.G.S.]; and start-up funds awarded to I.G.S.

References

[1] Scott Aaronson and Hendrik Kirchner. Watermarking GPT outputs. Talk slides, 2022. https://www.scottaaronson.com/talks/watermark.ppt.

[2] Masanori Arita and Yoshiaki Ohashi. Secret signatures inside genomic DNA. *Biotechnology Progress*, 20(5):1605–1607, 2004.

[3] David Baker and George Church. Protein design meets biosecurity. *Science*, 383(6681):349, 2024. Editorial.

[4] Loubna Ben Allal, Qiuyi Li, Maurizio Fiusco, Lewis Tunstall, Kashif Rasul, Ed Beeching, Dana Aubakirova, Carlos Patiño, Thibaud Frere, Anton Lozhkov, Georgia Channing, Thomas Wolf, Diego di Bernardo, and Leandro von Werra. Carbon: Decoding the language of life. *bioRxiv*, 2026. Preprint, version 1, posted 25 May 2026, doi:10.64898/2026.05.22.727119.

[5] Yanshuo Chen, Zhengmian Hu, Yihan Wu, Ruibo Chen, Yongrui Jin, Marcus Zhan, Chengjin Xie, Wei Chen, and Heng Huang. Enhancing privacy in biosecurity with watermarked protein design. *Bioinformatics*, 41(7):btaf141, 2025.

[6] Miranda Christ and Sam Gunn. Pseudorandom error-correcting codes. In *Advances in Cryptology — CRYPTO 2024*, Lecture Notes in Computer Science, pages 325–347. Springer, 2024. IACR ePrint 2024/235.

[7] Miranda Christ, Sam Gunn, and Or Zamir. Undetectable watermarks for language models. In *Proceedings of the 37th Annual Conference on Learning Theory (COLT)*, volume 247 of *PMLR*, pages 1125–1139, 2024. arXiv:2306.09194.

[8] Hugo Dalla-Torre, Liam Gonzalez, Javier Mendoza-Revilla, Nicolas Lopez Carranza, Adam Henryk Grzywaczewski, Francesco Oteri, Christian Dallago, Evan Trop, Bernardo P. de Almeida, et al. Nucleotide Transformer: building and evaluating robust foundation models for human genomics. *Nature Methods*, 22(2):287–297, 2025.

[9] Sumanth Dathathri, Abigail See, Sumedh Ghaisas, Po-Sen Huang, Rob McAdam, Johannes Welbl, Vandana Bachani, Alex Kaskasoli, Robert Stanforth, Tatiana Matejovicova, Jamie Hayes, Nidhi Vyas, Majd Al Merey, Jonah Brown-Cohen, Rudy Bunel, Borja Balle, Taylan Cemgil, Zahra Ahmed, Kitty Stacpoole, Ilia Shumailov, Ciprian Baetu, Sven Gowal, Demis Hassabis, and Pushmeet Kohli. Scalable watermarking for identifying large language model outputs. *Nature*, 634(8035):818–823, 2024.

[10] Matthew C. Davey and David J. C. MacKay. Reliable communication over channels with insertions, deletions, and substitutions. *IEEE Transactions on Information Theory*, 47(2):687–698, 2001.

[11] Abdulrahman Diaa, Toluwani Aremu, and Nils Lukas. Optimizing adaptive attacks against watermarks for language models. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, volume 267 of *PMLR*, pages 13546–13569, 2025.

[12] GenerTeam. GENERator-v2-eukaryote-1.2b-base model card and released modeling code. https://huggingface.co/GenerTeam/GENERator-v2-eukaryote-1.2b-base, 2026.

[13] Daniel G. Gibson, John I. Glass, Carole Lartigue, Vladimir N. Noskov, Ray-Yuan Chuang, Mikkel A. Algire, Gwynedd A. Benders, Michael G. Montague, Li Ma, et al. Creation of a bacterial cell controlled by a chemically synthesized genome. *Science*, 329(5987):52–56, 2010.

[14] Noah Golowich and Ankur Moitra. Edit distance robust watermarks via indexing pseudorandom codes. In *Advances in Neural Information Processing Systems 37 (NeurIPS)*, volume 37, pages 20645–20693. Curran Associates, Inc., 2024. arXiv:2406.02633.

[15] Google DeepMind. SynthID-Text reference implementation. https://github.com/google-deepmind/synthid-text, 2024. Commit addb4a15, 13 June 2025.

[16] Bernhard Haeupler and Amirbehshad Shahrasbi. Synchronization strings: Codes for insertions and deletions approaching the Singleton bound. In *Proceedings of the 49th Annual ACM SIGACT Symposium on Theory of Computing (STOC)*, pages 33–46, 2017.

[17] Xia Han, Qi Li, Jianbing Ni, and Mohammad Zulkernine. Robustness assessment and enhancement of text watermarking for Google's SynthID, 2025. arXiv:2508.20228; accepted at IEEE TrustCom 2025.

[18] Dominik Heider and Angelika Barnekow. DNA-based watermarks using the DNA-Crypt algorithm. *BMC Bioinformatics*, 8:176, 2007.

[19] Abe Hou, Jingyu Zhang, Tianxing He, Yichen Wang, Yung-Sung Chuang, Hongwei Wang, Lingfeng Shen, Benjamin Van Durme, Daniel Khashabi, and Yulia Tsvetkov. SemStamp: A semantic watermark with paraphrastic robustness for text generation. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 4067–4082. Association for Computational Linguistics, 2024.

[20] Zhengmian Hu, Lichang Chen, Xidong Wu, Yihan Wu, Hongyang Zhang, and Heng Huang. Unbiased watermark for large language models. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.10669.

[21] HuggingFaceBio. Carbon-500M model card. https://huggingface.co/HuggingFaceBio/Carbon-500M, 2026.

[22] Nikola Jovanović, Robin Staab, and Martin Vechev. Watermark stealing in large language models. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, volume 235 of *PMLR*, pages 22570–22593, 2024. arXiv:2402.19361.

[23] Samuel H. King, Claudia L. Driscoll, David B. Li, Daniel Guo, Aditi T. Merchant, Garyk Brixi, Max E. Wilkinson, and Brian L. Hie. Generative design of bacteriophages with genome language models. *Science*, 393(6811):eaec2657, 2026. PMID: 42561074.

[24] John Kirchenbauer, Jonas Geiping, Yuxin Wen, Jonathan Katz, Ian Miers, and Tom Goldstein. A watermark for large language models. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, volume 202 of *PMLR*, pages 17061–17084, 2023. arXiv:2301.10226.

[25] John Kirchenbauer, Jonas Geiping, Yuxin Wen, Manli Shu, Khalid Saifullah, Kezhi Kong, Kasun Fernando, Aniruddha Saha, Micah Goldblum, and Tom Goldstein. On the reliability of watermarks for large language models. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2306.04634.

[26] Rohith Kuditipudi, John Thickstun, Tatsunori Hashimoto, and Percy Liang. Robust distortion-free watermarks for language models. *Transactions on Machine Learning Research*, 2024. arXiv:2307.15593.

[27] Qiuyi Li, Zhihao Zhan, Shikun Feng, Yiheng Zhu, Yuan He, Wei Wu, Zhenghang Shi, Shengjie Wang, Zongyong Hu, Zhao Yang, Jiaoyang Li, Jian Tang, Haiguang Liu, and Tao Qin. GENERator-v2: Reconciling coarse tokenization with single-nucleotide resolution in genomic language modeling. *bioRxiv*, 2026. Preprint, version 2, posted 4 May 2026, doi:10.64898/2026.01.27.702015.

[28] Michael Liss, Daniela Daubert, Kathrin Brunner, Kristina Kliche, Ulrich Hammes, Andreas Leiherer, and Ralf Wagner. Embedding permanent watermarks in synthetic genes. *PLoS ONE*, 7(8):e42465, 2012.

[29] Zicheng Liu, Jiahui Li, Siyuan Li, Zelin Zang, Cheng Tan, Yufei Huang, Yajing Bai, and Stan Z. Li. GenBench: A benchmarking suite for systematic evaluation of genomic foundation models, 2024. arXiv:2406.01627.

[30] Frederikke Isa Marin, Felix Teufel, Marc Horlacher, Dennis Madsen, Dennis Pultz, Ole Winther, and Wouter Boomsma. BEND: Benchmarking DNA language models on biologically meaningful tasks. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2311.12570.

[31] Eric Nguyen, Michael Poli, Matthew G. Durrant, Brian Kang, Dhruva Katrekar, David B. Li, Liam J. Bartie, Armin W. Thomas, Samuel H. King, Garyk Brixi, et al. Sequence modeling and design from molecular to genome scale with Evo. *Science*, 386(6723):eado9336, 2024.

[32] Eric Nguyen, Michael Poli, Marjan Faizi, Armin Thomas, Michael Wornow, Callum Birch-Sykes, Stefano Massaroli, Aman Patel, Clayton Rabideau, Yoshua Bengio, Stefano Ermon, Christopher Ré, and Stephen Baccus. HyenaDNA: Long-range genomic sequence modeling at single nucleotide resolution. In *Advances in Neural Information Processing Systems 36 (NeurIPS)*, pages 43177–43201, 2023. arXiv:2306.15794.

[33] Romina Omidi, Yun Dong, and Binghui Wang. On Google's SynthID-Text LLM watermarking system: Theoretical analysis and empirical validation, 2026. arXiv:2603.03410.

[34] Aman Patel, Arpita Singhal, Austin Wang, Anusri Pampari, Maya Kasowski, and Anshul Kundaje. DART-Eval: A comprehensive DNA language model evaluation benchmark on regulatory DNA. In *Advances in Neural Information Processing Systems 37: Datasets and Benchmarks Track*, volume 37, pages 62024–62061, 2024. arXiv:2412.05430.

[35] Shayleen Reynolds, Hengzhi He, Dung Daniel T. Ngo, Saheed Obitayo, Niccolò Dalmasso, Guang Cheng, Vamsi K. Potluru, and Manuela Veloso. Breaking distortion-free watermarks in large language models, 2025.

[36] Vinu Sankar Sadasivan, Aounon Kumar, Sriram Balasubramanian, Wenxiao Wang, and Soheil Feizi. Can AI-generated text be reliably detected? stress testing AI text detectors under various attacks. *Transactions on Machine Learning Research*, 2025.

[37] Yair Schiff, Chia-Hsiang Kao, Aaron Gokaslan, Tri Dao, Albert Gu, and Volodymyr Kuleshov. Caduceus: Bi-directional equivariant long-range DNA sequence modeling. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, volume 235 of *PMLR*, pages 43632–43648, 2024. arXiv:2403.03234.

[38] Wei Wu, Qiuyi Li, Yuanyuan Zhang, Zhihao Zhan, Ruipu Chen, Mingyang Li, Kun Fu, Junyan Qi, Yongzhou Bao, Chao Wang, et al. GENERator: A long-context generative genomic foundation model. *arXiv preprint arXiv:2502.07272*, 2025. Version 5, 31 January 2026.

[39] Yihan Wu, Zhengmian Hu, Junfeng Guo, Hongyang Zhang, and Heng Huang. A resilient and accessible distribution-preserving watermark for large language models. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, volume 235 of *PMLR*, pages 53443–53470, 2024. arXiv:2310.07710.

[40] Hanlin Zhang, Benjamin L. Edelman, Danilo Francati, Daniele Venturi, Giuseppe Ateniese, and Boaz Barak. Watermarks in the sand: Impossibility of strong watermarking for language models. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, volume 235 of *PMLR*, pages 58851–58880, 2024. arXiv:2311.04378.

[41] Zaixi Zhang, Ruofan Jin, Le Cong, and Mengdi Wang. Securing the language of life: Inheritable watermarks from DNA language models to proteins. In *Advances in Neural Information Processing Systems 38 (NeurIPS)*, volume 38, pages 152050–152080, 2025. arXiv:2509.18207.

[42] Xuandong Zhao, Prabhanjan Ananth, Lei Li, and Yu-Xiang Wang. Provable robust watermarking for AI-generated text. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2306.17439.

Supplementary information

S1 Window-score selection

Before any detection result was examined, 64 development prompts were reserved for choosing how to summarize the mark bits in a window. Neither they nor the 192 held-out prompts were used to fit the detection threshold. The two candidates were the released SynthID mean score over scored tokens and the upstream default weighted-mean score. For GENERator, the two were compared on the 64 reserved prompts alone. The comparison used continuations of 384 and 3,072 bases, scored with the token phase known, and measured the separation between marked and ordinary continuations in units of the ordinary-continuation standard deviation. The mean score separated them at least as well at both lengths and was selected. No held-out prompt was scored during this selection, and no signal-matched weights were fitted. No other verifier setting was fitted to the reserved prompts: the window lengths and the significance level are fixed settings of the verifier, and the threshold follows from them and the sequence length. We applied the same mean score to Carbon, but the equivalent Carbon comparison is recorded only in the run notes, not as a retained result, so this selection is independently verified for GENERator only.

S2 Window geometry

Each scored token carries 30 mark bits, one per tournament layer. The number of bits does not limit detection: 21 bits that are all 1 already reach the corrected threshold, so one scored token would suffice. What limits detection is dilution. The shortest window, 384 bases, holds 64 tokens. The first four supply unwatermarked context, which leaves at most 60 scored tokens and at most 1,800 mark bits, fewer where a four-token context repeats. With all 60 scored, at least 1,004 of the 1,800 bits must be 1, about 56%. Longer windows need a smaller fraction. Detection therefore depends on recoverable synchronization, not on the absence of edits.

S3 Conservatism of the correction

Bonferroni correction bounds the family-wise error rate without requiring independence between tests. Because the searched windows are positively correlated, the effective number of independent tests is smaller than M. The realized false-positive rate can then lie well below α, and recall is attained under a stricter threshold than necessary. The realized per-sequence rates in the detection cohort (Section 4.3) are 9.5 times below α for Carbon and 5.4 times below for GENERator. The observed minimum window P-values behave like those of about 828–852 independent tests, consistently across both models and all four conditions. That is about 19 times fewer than the 16,136 windows searched. This moment-matched figure summarizes how conservative the correction is. It does not replace the decision rule, which is unchanged.

S4 Search cost and computing environment

Keyed hashing, not window enumeration, dominates the cost of the search. A sequence of N bases has twelve token frames, two orientations by six phases, and the mark bits of every token position in a frame are computed once, in a single pass. Window statistics then follow from prefix sums over those per-token bit counts, so each window of a given length is evaluated in constant time. Where a frame contains repeated four-token contexts, the first-occurrence rule is enforced with Fenwick trees, and each window costs $O(\log N)$ instead. The work is therefore $O(N)$ keyed hash evaluations and $O(N \log N)$ additional time, with $O(N)$ space per sequence, and M grows linearly in N for a fixed window set. Sequences shorter than every window length are not scored, reverse-complement coordinates are mapped back onto the supplied sequence, and log-probabilities are kept throughout to avoid floating-point underflow.

Timing was measured on an AMD EPYC 7763. One 3,456-base sequence took a median 0.0956 s on a single thread. Sequences of 6,912 bases took 0.245 s, and sequences of 13,824 bases took 0.902 s. Peak memory per process was 27.9 MiB for a 3,456-base sequence. For a 13,824-base sequence, it was 69.1 MiB. Running 16 workers in parallel across sequences gave one sequence per 0.0167 s of wall-clock time. That is 5.7 times the single-threaded rate.

Development-cohort generation ran on GPUs. Model scoring ran on CPU for Carbon and on GPU for GENERator. The detection cohort was generated on NVIDIA A100 GPUs in a separate computing environment. Detection ran on x86-64 CPUs for both cohorts.

S5 Development-cohort detection

Table S1 gives the development-cohort counts, and Figures S1 and S2 show strength and strongest-window length. Every marked sequence was found in both models and every condition. Each control produced at most one positive sequence per model and condition, and the GENERator wrong-key positive is absent after a deletion. With 192 prompts, the exact control intervals reach 2.868% for one positive prompt and 1.903% for none. This cohort therefore cannot show whether the false-positive rate lies below 1%; the detection cohort addresses that question.

Table S1: Positive detections in the development cohort, for each model across 192 held-out prompts with two draws each. Correct-key and wrong-key columns refer to marked sequences. Each edit changes exactly one base. The final two rows summarize unedited sequences at the prompt level to account for paired draws; intervals are exact and two-sided. All marked prompts also had both draws detected in every edit condition.

	Carbon-500M			**GENERator-v2 1.2B**		
Edit applied to the sequence	Correct key	Ordinary	Wrong key	Correct key	Ordinary	Wrong key
None	384/384	1/384	0/384	384/384	0/384	1/384
One substitution	384/384	1/384	0/384	384/384	0/384	1/384
One insertion	384/384	1/384	0/384	384/384	0/384	1/384
One deletion	384/384	1/384	0/384	384/384	0/384	0/384
Prompts with a positive draw	192/192	1/192	0/192	192/192	0/192	1/192
Exact 95% interval	98.10–100%	0.013–2.868%	0–1.903%	98.10–100%	0–1.903%	0.013–2.868%

S6 Strength and the weakest sequence

Typical strength is nearly the same in both models. In the development cohort, the median unedited marked sequence had strength 790.6 in Carbon and 798.8 in GENERator. Control strengths were concentrated near or below the threshold, and the few positive controls lay only slightly above it (Figure S1a,c). The weakest marked sequence differs more between the models. In the development cohort, it had strength 124.8 in Carbon and 19.6 in GENERator. In the detection cohort, the weakest sequences were 237.6 in Carbon and 123.8 in GENERator, and the medians were again close (Table S2). The difference between weakest sequences therefore reflects single sequences, not the models' typical margin.

The weakest detected GENERator sequence in the development cohort contained a highly repetitive continuation. Of its four-token contexts, 174 repeated and were excluded from scoring, which left 334 of a possible 508 tokens. Only 54.6% of its mark bits were 1, against a median of 74% in both models. Restoring all 508 scored tokens at that fraction would close only 9% of the gap to Carbon's weakest sequence, so most of the shortfall comes from weak signal in the tokens that remained. The tournament can raise the fraction of 1s only where the model spreads probability across several tokens, which is consistent with a low-entropy, repetitive stretch carrying little signal. We did not measure per-token entropy to confirm this. We therefore find no evidence that typical detection strength differs between the models.

S7 Aligned-detector diagnostic in GENERator

For GENERator, a separate diagnostic checked the analytic null with a detector given the known token phase. Across the four evaluated lengths, 2, 6, 6, and 4 ordinary prompts were positive, against about five expected at each length ($P = 0.25$–0.87). This result is compatible with the analytic null, but it does not replace the full-search correction used for the position-independent decision, and it is not a calibration of the full search (Section 6).

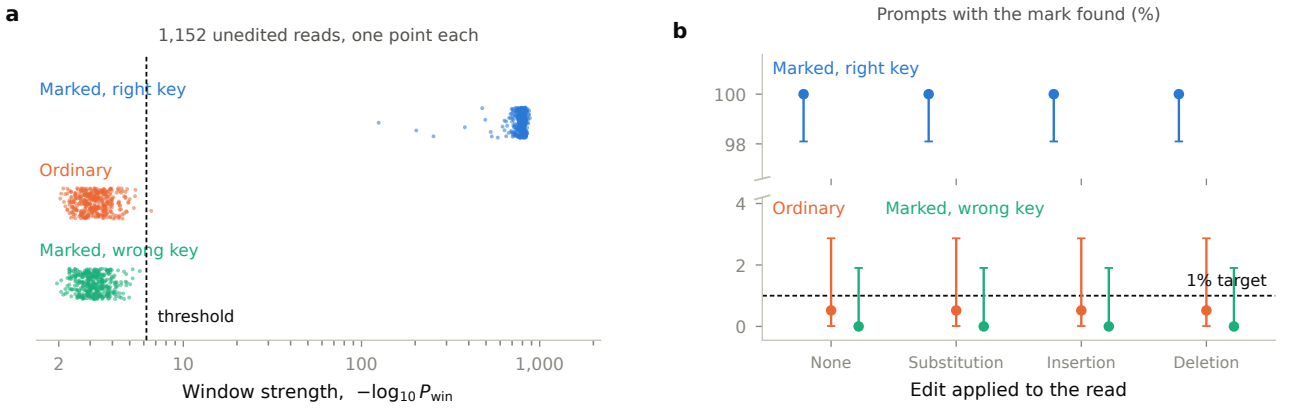
S8 Strength and window length by edit

Table S2 gives the median strength of marked sequences in the detection cohort and, for both cohorts, how often the full 3,072-base window carried the strongest evidence.

Table S2: Strength and strongest-window length of marked sequences, by edit. Median strength is for the detection cohort, over 3,088 sequences per model and condition. The full-window columns count sequences whose strongest window was the full 3,072-base window, out of 384 per model and condition in the development cohort and 3,088 in the detection cohort. Each edit changes exactly one base.

	Median strength		**Full window, development**		**Full window, detection**	
Edit applied to the sequence	Carbon	GENERator	Carbon	GENERator	Carbon	GENERator
None	792.9	799.7	383	381	3,087	3,085
One substitution	777.6	783.8	383	381	3,087	3,085
One insertion	441.8	443.1	203	200	1,724	1,732
One deletion	437.7	438.5	232	227	1,663	1,671

Carbon-500M



GENERator-v2 1.2B

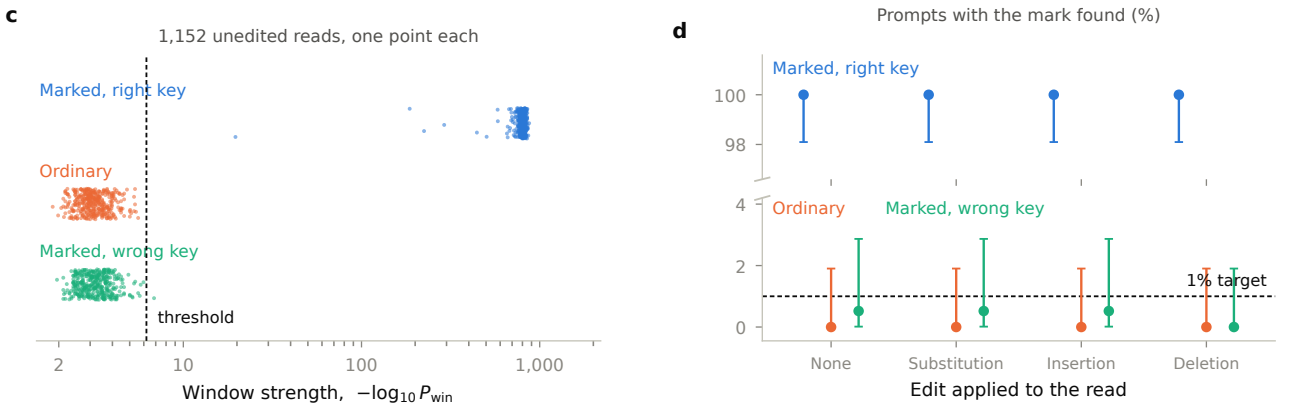


Figure S1: Position-independent detection in the development cohort. **a,c**, Strength of the best window in each unedited sequence, one point per sequence, on a log scale; in the panel labels, "Marked, right key" denotes marked sequences scored with the correct key, and "reads" denotes test sequences. The dashed line is the threshold corrected for the search over 16,136 windows. All marked sequences exceed the threshold. **b,d**, Proportion of the 192 held-out prompts with positive detections, with exact two-sided 95% intervals, on a broken axis. All marked prompts were detected in all four edit conditions. The GENERator wrong-key positive is absent after a deletion. With 192 prompts, the control intervals extend above the 1% target; Table 1 and Figure 3 report the detection cohort. Panels **a,b** show Carbon and panels **c,d** show GENERator.

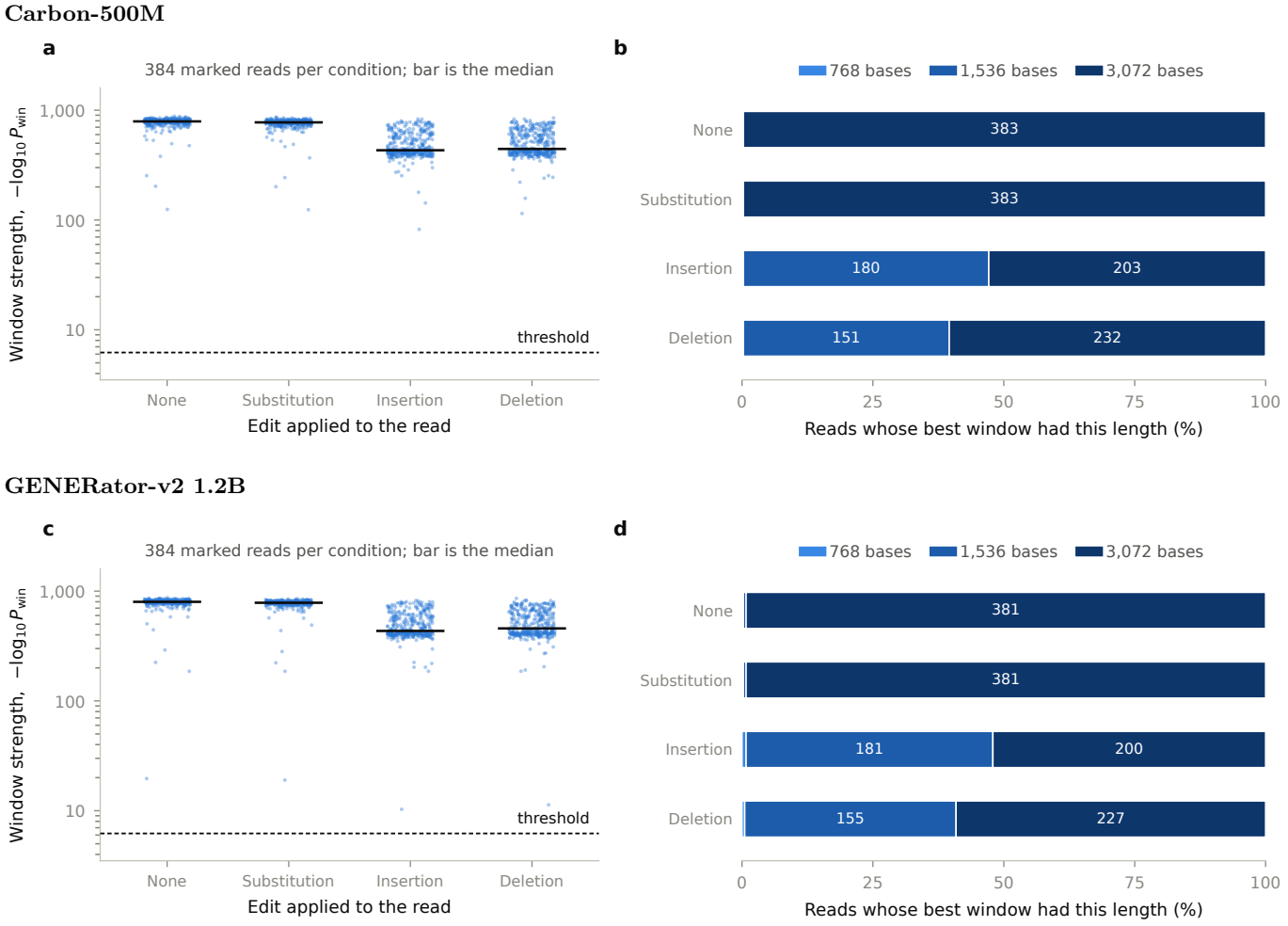


Figure S2: Single-base edits in the development cohort. In the panel labels, "reads" denotes test sequences. **a,c**, Strength of the best window in each of the 384 marked sequences per condition, on a log scale, with the median shown as a bar. The dashed line is the detection threshold. **b,d**, Length of the strongest window; numbers inside the bars are sequence counts out of 384. As in the detection cohort (Figure 4), the full 3,072-base window is strongest for nearly all unedited and substituted sequences, whereas shorter windows are more frequent after an insertion or deletion. Panels **a,b** show Carbon and panels **c,d** show GENERator.